

EVENTOSMADRID

Documentación Técnica del Proyecto

Versión
1.0 MVP

Año
2026

Plataforma
Web

Estado
En desarrollo

¿Qué es este documento?

Esta documentación explica cómo está construida la aplicación EventosMadrid: qué tecnologías utiliza, cómo están organizados los archivos, cómo fluyen los datos y qué hace cada parte del sistema. Está escrita para que cualquier persona —técnica o no— pueda entender el proyecto y su funcionamiento.

1. ¿Qué es EventosMadrid?

EventosMadrid es una aplicación web que funciona en el navegador y permite descubrir eventos culturales y de ocio en la Comunidad de Madrid. En vez de visitar múltiples páginas de agendas por separado, el usuario accede a un único lugar donde toda la información ya está reunida, filtrada y presentada de forma visual.

¿Qué puede hacer el usuario?

- Buscar eventos por nombre, categoría, zona o precio.
- Ver los eventos en un mapa interactivo, en una lista o en un calendario mensual.
- Guardar eventos favoritos para consultarlos más tarde.
- Planificar una ruta optimizada con varios eventos en un mismo día.
- Cambiar el idioma de la interfaz (disponible en 10 idiomas).
- Activar modos de accesibilidad: alto contraste, texto grande, sin animaciones.
- Ver estadísticas de la oferta cultural disponible.

Idea clave

El valor no está solo en mostrar información, sino en convertirla en algo accionable: qué ver, cuándo, dónde y cómo llegar.

2. Tecnologías utilizadas

El proyecto usa tecnologías estándar y ampliamente conocidas en el mundo del desarrollo web. A continuación se explica cada una con su función concreta dentro del proyecto:

2.1 Frontend (lo que ve el usuario)

Tecnología	Para qué sirve	Dónde se usa en el proyecto
HTML5	Lenguaje que define la estructura de una página web (qué elementos existen y dónde están).	index.html — define todos los paneles, botones y secciones de la interfaz.
CSS3	Lenguaje para dar estilo visual: colores, tipografías, disposición, animaciones.	Carpeta css/ — controla el aspecto de cada componente.
JavaScript (ES6+)	Lenguaje de programación que hace la web interactiva. Es el «cerebro» de la aplicación.	js/main.js — gestiona toda la lógica de usuario: filtros, mapa, calendario, favoritos...
Leaflet.js	Librería JavaScript para mostrar mapas interactivos en el navegador.	Carga el mapa de Madrid con los marcadores de cada evento.
Chart.js	Librería para crear gráficos y visualizaciones de datos.	Panel de estadísticas: distribución de eventos por tipo, zona y fecha.
Vite	Herramienta de desarrollo que acelera la carga del proyecto en local y genera la versión final optimizada.	vite.config.js — configura el entorno de desarrollo y la compilación.
Service Worker (sw.js)	Tecnología del navegador que permite funcionar sin conexión y acelera la carga guardando recursos en caché.	sw.js — intercepta peticiones y sirve recursos desde caché cuando no hay red.

2.2 Backend de datos / ETL

ETL son las siglas en inglés de Extraer, Transformar y Cargar. Es el proceso automático que obtiene los datos de eventos de fuentes externas, los limpia y los guarda para que el frontend los pueda usar.

Tecnología	Para qué sirve	Dónde se usa en el proyecto
Python 3	Lenguaje de programación muy usado para automatización y procesamiento de datos.	scraper/scraper.py — todo el proceso de obtención y limpieza de datos.
API ES Madrid	Fuente oficial de datos del Ayuntamiento de Madrid con eventos culturales.	scraper.py extrae eventos culturales de esta API pública.
API Ticketmaster	Plataforma de venta de entradas que expone una API con eventos de ocio.	scraper.py extrae conciertos, espectáculos y eventos de entretenimiento.
Nominatim (OpenStreetMap)	Servicio de geocodificación gratuito: convierte una dirección en coordenadas de latitud/longitud.	Cuando un evento no tiene coordenadas, se calculan automáticamente.
JSON	Formato estándar de intercambio de datos, fácil de leer tanto por humanos como por máquinas.	data/eventos.json y data/lugares.json — almacenan todos los datos del proyecto.

2.3 Automatización e infraestructura

Tecnología	Para qué sirve	Dónde se usa en el proyecto
GitHub Actions	Sistema de automatización integrado en GitHub que ejecuta tareas programadas.	.github/workflows/scraper.yml — ejecuta el scraper periódicamente y actualiza los datos.
Node.js / npm	Entorno de ejecución de JavaScript fuera del navegador, necesario para instalar dependencias.	Usado para instalar y ejecutar Vite y otras herramientas de desarrollo.
localStorage	Almacenamiento del propio navegador del usuario, sin necesidad de base de datos.	Guarda los favoritos y las preferencias del usuario entre sesiones.

3. Estructura del proyecto

El código está organizado en carpetas que agrupan archivos por su función. Aquí se explica qué contiene cada parte:

Archivo / Carpeta	Qué contiene y para qué sirve
<code>index.html</code>	Página principal de la aplicación. Define toda la interfaz visual: el mapa, los filtros, los paneles, el menú y todos los botones. Es el esqueleto de lo que ve el usuario.
<code>js/main.js</code>	El corazón del proyecto. Contiene toda la lógica: cómo se cargan y filtran los eventos, cómo funciona el mapa, el calendario, los favoritos y la ruta. Es el archivo más importante y extenso.
<code>js/i18n.js</code>	Motor de internacionalización (i18n). Detecta el idioma del navegador, carga el diccionario correspondiente y traduce todos los textos de la interfaz de forma dinámica.
<code>js/locales/</code>	Carpeta con los archivos de traducción: <code>es.js</code> (español), <code>en.js</code> (inglés), <code>fr.js</code> (francés), <code>pt.js</code> (portugués), <code>de.js</code> (alemán), <code>it.js</code> (italiano), <code>zh.js</code> (chino), <code>ja.js</code> (japonés), <code>ko.js</code> (coreano), <code>ar.js</code> (árabe).
<code>js/modules/store.js</code>	Estado global de la aplicación. Guarda en memoria todos los datos que la app necesita mientras el usuario navega: qué vista está activa, qué filtros están aplicados, cuáles son los eventos cargados...
<code>js/modules/constants.js</code>	Catálogo de valores fijos: los iconos y colores de cada categoría de evento, y las coordenadas geográficas de las zonas de Madrid (Centro, Retiro, Chamberí...).
<code>css/</code>	Estilos visuales de toda la interfaz: colores, fuentes, márgenes, diseño responsivo para móvil y escritorio, y efectos visuales.
<code>data/eventos.json</code>	Base de datos de eventos en formato JSON. La actualiza el scraper automáticamente y la lee el frontend al cargar la página.
<code>data/lugares.json</code>	Lista de lugares de interés complementarios (museos, parques, teatros...) que se muestran en el mapa junto a los eventos.
<code>scraper/scraper.py</code>	Script de Python que obtiene eventos de las APIs de ES Madrid y Ticketmaster, los normaliza, geocodifica, elimina duplicados y guarda el resultado en <code>data/eventos.json</code> .
<code>scraper/config.py</code>	Configuración del scraper: qué fuentes consultar, qué palabras clave definen cada categoría, qué municipios y venues conocidos existen en Madrid.
<code>.github/workflows/</code>	Automatización CI/CD: define cuándo y cómo se ejecuta el scraper de forma programada (sin intervención manual) y cómo se publican los datos actualizados.
<code>sw.js</code>	Service Worker: gestiona la caché del navegador para que la app funcione con conexión lenta o intermitente, y para acelerar las cargas repetidas.
<code>vite.config.js</code>	Configuración de Vite: define cómo se compila el proyecto para producción y cómo funciona el servidor local durante el desarrollo.

4. Cómo está organizado el sistema (capas)

El sistema está dividido en siete capas, cada una con una responsabilidad diferente. Comparado con las distintas partes de un restaurante sería: el salón (lo que ve el cliente), la cocina (la lógica), el almacén (los datos), y el proveedor externo (scraper).

#	Capa	Qué hace	Archivos clave
1	Presentación	Define la interfaz visual y los elementos con los que interactúa el usuario (botones, paneles, formularios de búsqueda).	index.html, css/
2	Lógica cliente	Gestiona todas las interacciones: filtros, mapa, calendario, favoritos, planificador de rutas, estadísticas.	js/main.js
3	Estado y catálogos	Almacena en memoria el estado actual de la aplicación y los valores fijos de dominio (iconos, zonas, colores).	store.js, constants.js
4	Internacionalización	Detecta el idioma del usuario y traduce la interfaz de forma dinámica. Tiene soporte para 10 idiomas.	js/i18n.js, js/locales/
5	Datos	Archivos JSON que contienen los eventos y lugares disponibles para renderizar en la interfaz.	data/eventos.json, data/lugares.json
6	ETL externo	Proceso automatizado que obtiene datos de APIs externas, los limpia y los guarda listos para el frontend.	scraper/*.py, workflow
7	Offline / Rendimiento	Mejora la experiencia en redes lentas y permite un uso básico sin conexión a internet.	sw.js

5. Cómo fluye la información

5.1 Cuando el usuario abre la aplicación

Esto es lo que ocurre, paso a paso, desde que el usuario escribe la URL en el navegador:

- El navegador descarga index.html con la estructura visual.
- Se detecta el idioma del sistema y se carga el diccionario de traducción correspondiente.
- Se ejecuta main.js, que prepara el estado de la aplicación y activa todos los listeners de eventos de usuario.
- Se cargan los archivos data/eventos.json y data/lugares.json con toda la información disponible.
- Se inicializa el mapa Leaflet con los marcadores de eventos. Si el usuario acepta, se activa la geolocalización.
- El Service Worker se registra en segundo plano para gestionar la caché de forma transparente.
- La aplicación queda lista: el usuario puede buscar, filtrar, explorar el mapa y guardar favoritos.

5.2 Cuando el usuario filtra o busca

Cada vez que el usuario cambia un filtro (tipo de evento, zona, precio, fecha) o escribe en el buscador, ocurre lo siguiente:

- La función applyFilters() en main.js recalcula qué eventos cumplen todos los criterios activos.
- La vista activa (mapa, lista o calendario) se actualiza instantáneamente sin recargar la página.
- Los marcadores del mapa, las tarjetas de la lista y los días del calendario reflejan el nuevo conjunto filtrado.

5.3 El ciclo automático de actualización de datos

Los datos de eventos se renuevan de forma completamente automática sin necesidad de intervención manual:

- GitHub Actions ejecuta el scraper según un calendario programado (por ejemplo, cada noche).
- scraper.py consulta las APIs de ES Madrid y Ticketmaster para obtener eventos nuevos.
- Normaliza los datos: unifica nombres de campos, limpia textos, asigna categorías por palabras clave.
- Geocodifica los eventos sin coordenadas usando Nominatim (convierte direcciones en latitud/longitud).
- Elimina duplicados y eventos cuya fecha ya ha pasado.
- Guarda el resultado en data/eventos.json. La próxima vez que alguien abra la app, verá datos frescos.

Sin este ciclo automático

Los datos de eventos quedarían desactualizados. Este pipeline garantiza que la información sea siempre relevante y válida sin trabajo manual continuo.

6. Funcionalidades principales

En la siguiente tabla se describe cada funcionalidad desde el punto de vista del usuario, junto con la tecnología que la hace posible:

Funcionalidad	Qué hace (para el usuario)	Tecnología implicada
Búsqueda y filtros combinados	Permite encontrar eventos relevantes en segundos combinando múltiples criterios: tipo, zona, precio y fecha.	JavaScript (main.js), HTML inputs
Mapa con clustering	Muestra los eventos en un mapa geográfico. Cuando hay muchos puntos juntos, los agrupa para no saturar la vista.	Leaflet.js + plugin MarkerCluster
Vista lista	Presenta los eventos como tarjetas con imagen, descripción, precio y acciones directas (guardar, compartir, ruta).	JavaScript, HTML/CSS
Vista calendario	Proyecta los eventos sobre un calendario mensual interactivo. Al pulsar un día aparecen los eventos de esa fecha.	JavaScript (renderCalendar)
Favoritos persistentes	El usuario puede marcar eventos como favoritos. Esta selección se conserva aunque cierre el navegador.	localStorage del navegador
Planificador de rutas	El usuario selecciona varios eventos y la app calcula el orden más eficiente para visitarlos en ciudad.	Leaflet.js + lógica en main.js
Internacionalización (i18n)	La interfaz completa está disponible en 10 idiomas. El cambio es instantáneo y sin recargar la página.	js/i18n.js + diccionarios js/locales/
Accesibilidad visual	Modo alto contraste, ajuste de tamaño de texto y desactivación de animaciones para usuarios con necesidades especiales.	CSS variables, JavaScript
Dashboard de estadísticas	Gráficos con la distribución de eventos por tipo, zona y franja horaria sobre los datos actualmente visibles.	Chart.js
Funcionamiento offline	Recursos estáticos (HTML, CSS, JS) cacheados para que la app cargue aunque la conexión sea lenta o caída.	Service Worker (sw.js)
Actualización automática de datos	Los eventos se actualizan sin intervención humana mediante un proceso programado en la nube.	Python, GitHub Actions

7. Evaluación técnica del proyecto

Esta sección ofrece una valoración objetiva del estado actual del código: qué funciona bien y qué se puede mejorar.

✓ Fortalezas

Producto funcional de extremo a extremo

El sistema cubre todo el ciclo: obtención de datos → procesamiento → experiencia de usuario. No hay partes incompletas.

Cobertura funcional amplia

Incluye mapa, lista, calendario, favoritos, rutas, estadísticas e internacionalización en un solo producto.

Resiliencia ante pérdida de red

El Service Worker garantiza que la app siga funcionando aunque el usuario pierda conexión.

ETL con saneamiento real

El scraper no solo descarga datos: los valida, geocodifica, deduplica y categoriza. Los datos que llegan al frontend son de calidad.

Pipeline completamente automatizado

La actualización de datos no requiere acción manual, reduciendo el coste operativo.

⚠ Aspectos a mejorar

main.js es un archivo monolítico

Actualmente concentra demasiadas responsabilidades en un único archivo largo. Esto dificulta leerlo, modificarlo y probarlo. La solución es dividirlo en módulos más pequeños (uno para el mapa, otro para filtros, otro para el calendario, etc.).

Lógica de negocio y de UI acopladas

La lógica de qué filtrar y la lógica de cómo mostrarlo están mezcladas. Separarlas facilita el mantenimiento y permite cambiar la interfaz sin tocar las reglas de negocio.

Ausencia de tests automatizados

No existen pruebas automáticas que verifiquen que el código funciona correctamente tras cambios. Añadir tests unitarios protegería contra regresiones.

Manejo de errores en integraciones externas

Las conexiones con APIs externas (Ticketmaster, ES Madrid, Nominatim) no tienen contratos ni manejo de errores estandarizado, lo que puede causar fallos silenciosos.

8. Conclusión

EventosMadrid es una base técnica sólida con enfoque de producto real. No se limita a mostrar información: la transforma en decisiones concretas para el usuario — qué ver, cuándo, dónde y cómo llegar.

El stack tecnológico elegido (JavaScript puro, Leaflet, Chart.js, Python y GitHub Actions) es maduro, bien documentado y sin dependencias innecesarias. La arquitectura actual ya entrega valor. Su próximo salto de calidad pasa por tres áreas:

- Modularización de main.js para mejorar mantenibilidad.
- Incorporación de tests automatizados para garantizar estabilidad ante cambios.
- Estandarización del manejo de errores en las integraciones con APIs externas.

Resumen en una frase

El proyecto tiene los cimientos bien puestos. Las mejoras propuestas no son urgentes, pero harán que el código sea más fácil de mantener, ampliar y colaborar a medida que el equipo o la funcionalidad crezcan.